Discrimination of Objects Mounted on Optical Bench Using Ultrasonic Sensing and Machine Learning

Rashid, Romana
M.Eng. in Information Technology
Frankfurt University of Applied Sciences
Frankfurt, Germany
romana.rashid@stud.fra-uas.de
Matriculation No. 1428733

Prabhu, Mithila Maruti
M.Eng. in Information Technology
Frankfurt University of Applied Sciences
Frankfurt, Germany
mithila.prabhu@stud.fra-uas.de
Matriculation No. 1567111

Abstract*—This paper presents an experimental study on the automated discrimination of objects mounted on an optical bench using a Red Pitaya ultrasonic sensing system combined with supervised machine learning classifiers. The objects under test represent a range of surface geometries and acoustic properties, including metallic, rigid plastic, foam-covered, and hollow plastic surfaces. Ultrasonic echo waveforms are acquired across a range of distances using the Red Pitaya platform, with multiple ADC measurements collected per acquisition to ensure signal stability. A multi-dimensional feature vector is extracted from each waveform, combining time-domain statistics such as mean amplitude, standard deviation, root mean square, and signal energy with spectral features derived from the Fast Fourier Transform. Three machine learning classifiers are trained on a dataset acquired on the first measurement day: Support Vector Machine, Random Forest, and K-Nearest Neighbours. The trained models are serialised with timestamps and subsequently reloaded to evaluate their performance on a fresh dataset acquired on a separate day without retraining. A secondary experiment trains and evaluates classifiers using measurements taken only at a small number of selected distances rather than the full measurement range. Results demonstrate that Random Forest achieves the highest in-session classification accuracy, which declines substantially when the model is applied to data from the second day, revealing significant temporal drift caused by environmental variation in the ultrasonic signal characteristics. When retrained on data from only the selected discrete distances, the Random Forest model recovers a notably higher cross-day accuracy, outperforming both SVM and KNN under all experimental conditions. The study confirms that ultrasonic classifiers are susceptible to inter-session performance degradation and identifies Random Forest combined with targeted distance-specific training as the most temporally stable configuration for this application. Keywords— Object discrimination, ultrasonic sensing, optical bench measurements, feature extraction, machine learning, Random Forest

I. Introduction

Object discrimination using sensor signals is an important task in intelligent sensing and automated measurement systems. In applications such as robotics, industrial automation, and smart sensing environments, machines must be able to recognize and differentiate objects based on signals obtained from sensors. When an object interacts with a sensing device, the resulting signal contains information about its physical characteristics, including material properties, surface structure, and distance from the sensor. By analyzing these signals, objects can be automatically identified and classified without human intervention [1].

Recent advances in signal processing and machine learning have significantly improved the analysis of complex sensor data. Traditional signal processing techniques often rely on manually designed rules or threshold-based methods, which may not perform well when signals vary due to environmental conditions or object variability. Machine learning provides a more flexible approach by allowing algorithms to learn patterns directly from labeled datasets [2]. Once trained, these models can classify new signals based on the relationship between signal features and object classes.

Ultrasonic sensing is widely used for non-contact object detection and measurement. Ultrasonic sensors emit high-frequency acoustic waves that propagate through the surrounding medium. When these waves encounter an object, a portion of the signal is reflected back toward the sensor. The characteristics of the reflected signal depend on factors such as the material of the object, surface structure, orientation relative to the sensor, and the distance between the sensor and the object. These variations in reflected signals can therefore be used to discriminate between different objects [3].

In this project, ultrasonic signals were recorded from objects mounted on an optical bench using a Red Pitaya measurement platform. Red Pitaya is a compact open-source system that integrates high-resolution analogue-to-digital converters and signal processing capabilities for acquiring and analyzing analog signals [4]. The optical bench provides a controlled experimental environment that allows precise positioning of objects relative to the ultrasonic sensor, ensuring consistent measurement conditions.

The recorded waveform signals were processed to extract meaningful information suitable for machine learning analysis. Statistical and frequency-domain features were obtained from each waveform to capture important signal characteristics while reducing data dimensionality [6]. These features were used to construct a dataset for supervised machine learning classification [5], [2]. Several classification algorithms, including Random Forest, Support Vector Machine, and K-Nearest Neighbors, were trained to discriminate between the objects based on their ultrasonic signal responses.

The main objective of this study is to investigate the effectiveness of machine learning techniques for discriminating objects using ultrasonic sensor measurements obtained from an optical bench experiment. By combining signal processing methods with machine learning algorithms, this work aims to develop a reliable approach for identifying objects based on their ultrasonic signal responses and to demonstrate the potential of machine learning methods in intelligent sensing systems.

II. Theoretical Background

A. Red Pitaya Ultrasonic System

Ultrasound (US) imaging is a common medical tool these days. It is feasible to construct an image of almost any organ or tissue in the human body by capturing the reflected signals of acoustic signal bursts at frequencies much above the human hearing limit, which are reflected by the various tissues with varying intensities. While this technology is undoubtedly comparable to other imaging methods like conventional radiography or CT scans, it also has the benefits of being less expensive and operating in a safer, radiation-free manner.

Red Pitaya is an open-source measurement and signal processing platform that integrates signal acquisition hardware, programmable logic, and software tools within a compact embedded system. The platform is widely used in research and engineering applications for capturing, processing, and analyzing analog signals. It provides high-resolution analog inputs, digital signal processing capabilities, and network connectivity, allowing real-time acquisition and analysis of sensor signals. In this project, Red Pitaya was used to acquire ultrasonic sensor signals and convert them into digital waveform data for further signal processing and machine learning analysis. The hardware architecture of Red Pitaya includes analog input channels, an analog-to-digital converter (ADC), programmable logic, and an embedded processor. The ultrasonic sensor output is connected to the analog input channel of the Red Pitaya system. Since sensor signals are analog in nature, the analog-to-digital converter samples the incoming signal and converts it into a digital waveform. This digital representation allows the signal to be stored, processed, and analyzed using computational methods. In this project, it is configured as an ultrasonic transceiver: the FPGA generates precisely timed transmit pulses to drive an ultrasonic transducer, while the received echo signal is digitized by a 14-bit analogue-to-digital converter (ADC) running at 125 Msps. The platform communicates with a host computer over a UDP network interface, and a dedicated GUI application (Frankfurt UAS, v0.23) controls acquisition parameters, displays the ADC and FFT waveforms in real time, and saves measurement files to disk. This setup provides a stable and reproducible environment for acquiring ultrasonic measurements from objects mounted on an optical bench. [4]

The measurement software allows the user to configure the number of ADC measurements per acquisition. In all experiments reported here, a count of 50 measurements is used. Each measurement returns one ADC waveform (8192 samples) followed by one FFT waveform (8192 samples), giving a total of 100 waveform blocks per saved file. The data loader used in this project parses these files by splitting the continuous sample stream into alternating ADC and FFT blocks starting from sample index 300, which skips the file header.

B. Analogue-to-Digital Conversion

The ADC is the hardware component responsible for converting the continuous-time analogue echo voltage at the ultrasonic receiver into a discrete-time digital sequence suitable for processing by the computer. The Red Pitaya ADC samples at 125 Msps with 14-bit resolution, providing a dynamic range of approximately 84 dB. Each captured waveform of 8192 samples represents approximately 65.5 microseconds of signal, sufficient to capture the primary echo from objects at distances up to approximately 11 metres in standard air condition. For the short distances tested in this project (5 to 30 cm), the echo arrives well within the first few thousand samples of each waveform. The time-of-flight between the transmitted pulse and the peak of the echo envelope is directly proportional to the object distance, providing the primary geometric cue for distance estimation. However, the shape and amplitude of the echo also encode information about the surface geometry and material of the object, which is the basis for object discrimination. The resulting digital signal preserves important characteristics such as amplitude, timing, and waveform shape, which are essential for further signal processing and feature extraction. [6]

C. Fast Fourier Transform Features

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT) of a discrete-time signal. It decomposes the time-domain waveform into its constituent frequency components, revealing the spectral content of the echo. For a real-valued signal of length N, the real FFT produces N/2 + 1 complex spectral coefficients, from which the magnitude spectrum is computed. In the context of ultrasonic echo analysis, the FFT is valuable because different objects and surface geometries produce echoes with characteristic frequency distributions. Hard, flat surfaces (such as the steel plate) tend to produce spectrally compact echoes concentrated near the transducer resonance frequency, while complex or absorbing surfaces (such as foam-covered objects) diffuse energy across a broader spectral range. The FFT magnitude spectrum therefore complements the time-domain features in the classification feature vector. In this project, five spectral features are extracted from the FFT magnitude of each ADC waveform: mean spectral amplitude, standard deviation of the spectrum, maximum spectral amplitude, the frequency bin index of the spectral peak, and the total spectral energy (sum of squared magnitudes). This transformation reveals the spectral content of the signal, including dominant frequencies and energy distribution. In ultrasonic sensing, different materials and surface structures produce characteristic frequency responses in the reflected signal [7].

D. Classifiers

1) Random Forest: Random Forest is an ensemble learning method that constructs a large number of decision trees during training and aggregates their predictions by majority voting [2]. Each tree is trained on a bootstrap sample of the training data, and at each split node, only a random subset of features is considered. This double randomization reduces variance and improves generalization relative to single decision trees. In this project, 500 trees are used with no maximum depth constraint, allowing the forest to grow until all leaves are pure.

Random Forest is well-suited to the ultrasonic classification task because it handles correlated features gracefully, provides implicit feature importance estimates, and its ensemble averaging naturally smooths out the effect of noisy individual waveform measurements. In the cross day experiments, Random Forest consistently outperforms SVM and KNN, suggesting that its variance-reduction properties also partially mitigate the effect of temporal distribution shift.

2) Support Vector Machine: Support Vector Machine (SVM) is a supervised learning algorithm that aims to find the optimal decision boundary that maximizes the separation between different classes [1]. As illustrated in Fig. X, SVM constructs a hyperplane that divides the feature space into distinct regions corresponding to each class, while maximizing the margin, which is the distance between the hyperplane and the closest data points from each class. The data points that lie closest to the decision boundary are known as support vectors, and they play a critical role in defining the position and orientation of the hyperplane. By focusing only on these critical samples, SVM is able to construct a robust classifier even in high-dimensional spaces. In cases where the data is not linearly separable, SVM employs the kernel trick to map the data into a higher-dimensional feature space where a linear separation becomes possible. In this project, a standard kernel (e.g., radial basis function or linear kernel) is used to model the relationship between ultrasonic features and object classes. SVM is particularly effective when: - The number of features is large relative to the number of samples - The classes are separable with a clear margin - The decision boundary is well-defined However, SVM has certain limitations in the context of ultrasonic classification. It is sensitive to: - Noise and overlapping class distributions - Choice of kernel and hyperparameters - Variations in data distribution (temporal drift) In this project, SVM achieves moderate performance on Day 1 but shows a significant drop in accuracy during cross-day evaluation. This indicates that SVM struggles to generalize when the data distribution changes between training and testing conditions. As observed in the results, the model is less robust compared to Random Forest, particularly under varying environmental conditions and sensor variability.

3) K-Nearest Neighbours: K-Nearest Neighbours (KNN) is a non-parametric, instance-based learning algorithm that performs classification based on local neighbourhood structure in the feature space. Unlike parametric models, KNN does not assume an explicit functional form for the decision boundary; instead, it relies directly on the distribution of the training data.

Given a query sample $\mathbf{x}$, the algorithm computes its distance to all training samples $\{\mathbf{x}_i\}_{i=1}^{N}$ using a metric $d(\mathbf{x}, \mathbf{x}_i)$, typically the Euclidean distance:

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{D} (x_j - x_{i,j})^2}$$

where D is the dimensionality of the feature vector. The K samples with the smallest distances form the local neighbourhood $\mathcal{N}_K(\mathbf{x})$.

The predicted class label $\hat{y}$ is determined by majority voting among these neighbours. In this work, a distance-weighted voting scheme is employed, where each neighbour contributes proportionally to the inverse of its distance:

$$\hat{y} = \arg\max_{c} \sum_{i \in \mathcal{N}_K(\mathbf{x})} \frac{\mathbb{I}(y_i = c)}{d(\mathbf{x}, \mathbf{x}_i) + \epsilon}$$

where $\mathbb{I}(\cdot)$ is the indicator function and ϵ is a small constant to avoid division by zero. This weighting ensures that closer neighbours exert greater influence on the final decision, improving robustness in locally dense regions of the feature space.

In this project, $K = 7$ was selected empirically to balance bias and variance: smaller values of K lead to high variance and sensitivity to noise, while larger values increase bias by smoothing class boundaries excessively. Feature scaling is critical for KNN because distance calculations are directly affected by feature magnitudes; therefore, all features are standardised using a `StandardScaler` prior to classification.

A key characteristic of KNN is the absence of an explicit training phase. The algorithm simply stores the entire training dataset and defers all computation to inference time, resulting in a computational complexity of $\mathcal{O}(ND)$ per query. While this makes KNN simple and flexible, it also introduces sensitivity to the structure, density, and distribution of the training data.

This sensitivity becomes a major limitation under distribution shift. In the present study, feature distributions change significantly between Day 1 (training) and Day 2 (testing) due to environmental variations and sensor repositioning. As a result, the geometric relationships in feature space are altered: samples that were previously close may become distant, and vice versa. Since KNN relies entirely on these local distances, such shifts distort the neighbourhood structure and lead to incorrect class assignments.

Furthermore, KNN is particularly vulnerable in high-dimensional and correlated feature spaces. Many of the extracted features in this work exhibit strong correlations ($r > 0.95$), which effectively reduces the intrinsic dimensionality while still affecting distance computations. This redundancy amplifies the impact of noisy or drifting features, such as `argmax` and `argmin`, further degrading performance under cross-session conditions.

Overall, while KNN provides a simple and interpretable baseline with competitive in-session performance, its dependence on stable feature geometry makes it poorly suited for

scenarios with significant temporal drift, as observed in the cross-day evaluation.

E. Summary

The theoretical concepts described above form the foundation of the proposed methodology. Signal acquisition through ADC, feature extraction using both time-domain and frequency-domain techniques, and classification using machine learning algorithms are combined to enable effective discrimination of objects based on ultrasonic measurements.

III. Experimental Setup

A. Hardware Configuration

The platform consists of a Red Pitaya STEMlab board connected to an ultrasonic transducer at one end of an aluminium optical bench. The test object is placed on a movable carrier that can be positioned at precisely defined distances. The optical bench enforces collinear alignment and keeps orientation fixed (face perpendicular to beam axis) throughout all measurements.

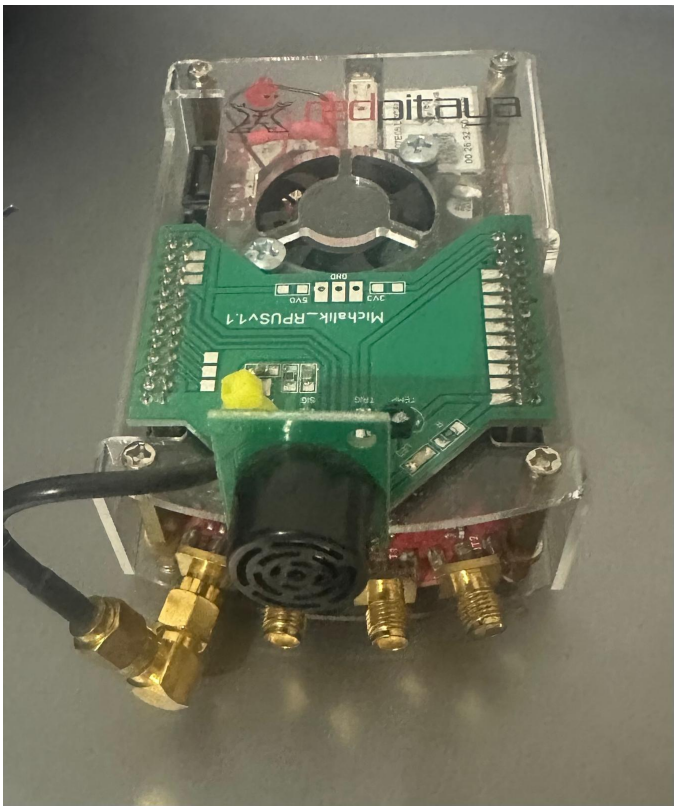

Fig. 1: Experimental setup — Red Pitaya STEMlab ultrasonic system on the optical bench with object under test.

B. Software Interface

The Frankfurt UAS UDP Client (v0.23) provides real-time visualisation of ADC and FFT waveforms. The operator sets the measure count to 50, initiates logging, and the application saves interleaved ADC/FFT data to text files. Each saved file contains five rows of measurement data in the format described in Section II.

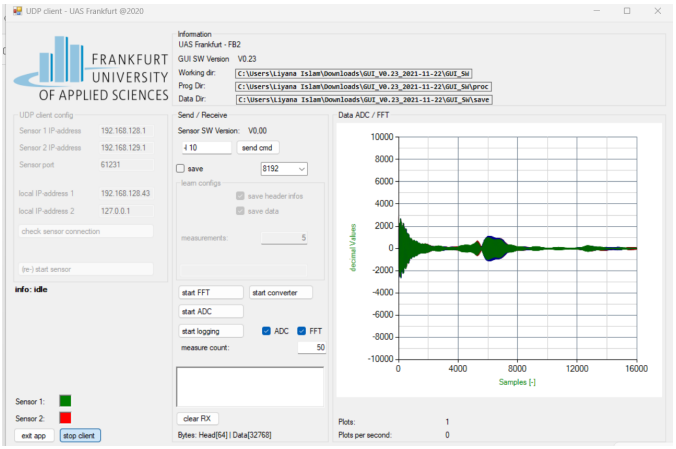

Fig. 2: Frankfurt UAS UDP Client GUI (V0.23) showing real-time ADC/FFT waveform display. Measure count set to 50.

C. Objects Under Test

Three physical objects were selected to cover a range of acoustic cross-sections and surface impedances (Table I). The plastic box (PB) was measured in two configurations — with and without an internal foam insert — yielding four distinct classes in total. Figure 3 shows the objects under test.

(a) Laptop **(b)** Steel plate **(c)** Plastic box

Fig. 3: Objects under test: laptop (smooth plastic lid), steel plate (rigid metal), and plastic box (tested with and without foam insert).

TABLE I: Objects Under Test and Expected Acoustic Properties

Object	Surface	Expected Echo	Discriminability
Laptop	Smooth plastic	Strong, compact	High
Steel plate	Rigid metal	Very strong, narrow	High
PB w/ Foam	Foam (absorbing)	Weak, broad	Moderate
PB w/o Foam	Hollow plastic	Moderate, diffuse	Low

D. Filename Convention and Label Assignment

Each raw data file encodes both the object class and distance in its name: the first character identifies the object (`l` = laptop, `s` = steel, `p` = plastic box), the numeric suffix gives the distance in centimetres (`l10.txt` = laptop at 10 cm, `s20.txt` = steel at 20 cm). PB w/foam and PB w/o foam share the prefix `p`; the two variants are distinguished by a subfolder-level label applied during dataset construction. The `source_file` column in the feature CSV preserves the original filename for per-file majority-vote evaluation.

E. Dataset Composition

Table II summarises the full dataset. Each distance from 5 to 30 cm was measured for each object on both days; each raw file contains five ADC measurements, yielding five feature vectors per file.

TABLE II: Dataset Composition — Samples per Class per Day

Class	Day 1	Day 2
Laptop	130	130
PB w/ Foam	145	135
PB w/o Foam	130	135
Steel	135	130
Total	**540**	**530**
Unique Files	**108**	**104**

F. Measurement Protocol

Day 1 measurements cover every integer distance from 5 to 30 cm (26 distances), four objects, with five ADC captures per file, yielding 540 feature vectors in total. Day 2 follows the identical spatial protocol, producing 530 feature vectors across 104 files. Both sessions were conducted in the same laboratory on different calendar days, introducing natural variation in ambient temperature, humidity, and sensor warm-up state.

The discrete-distance experiment uses only distances of 10, 15, 20, and 25 cm on both days — 95 training samples and 85 test samples respectively — to evaluate whether focusing on fewer measurement positions improves cross-day generalisation.

IV. Methodology

A. Data Loading and Parsing

Each raw file is parsed by reading its rows (each row is one measurement), skipping the first 16 header tokens, extracting the next 8,192 values as the ADC waveform, and discarding the trailing 8,192 pre-computed FFT values. The parser must handle non-numeric header tokens (e.g., version string `V0.2` and European-format decimal values such as `0,25`) by splitting on tab characters rather than relying on `numpy.loadtxt` directly.

```python
import numpy as np

with open(file_path, 'r') as fh:
    rows = fh.readlines()

adc_blocks = []
for row in rows:
    tokens = row.strip().split('\t')
    # Skip 16-value header, take next 8192 as ADC
    adc = [float(t) for t in tokens[16:16+8192]]
    adc_blocks.append(np.array(adc))
# tokens[16+8192:] would be the FPGA FFT     not used
```

Listing 1: Data loading and parsing (data_loader.py).

The distance label is derived from the filename suffix (e.g., `l10.txt` → 10 cm) and appended as the 16th feature. The object class label (`laptop`, `steel`, `plastic box with foam`, `plastic box without foam`) is assigned from the sub-directory in which the file resides.

B. Feature Extraction

Fifteen features are extracted from each ADC waveform and distance is appended as the 16th input (Table III). All features are computed from the raw ADC values; the embedded FPGA FFT block is not used.

TABLE III: Complete 16-Dimensional Feature Vector

#	Feature	Domain
1	Mean amplitude (μ)	Time
2	Standard deviation (σ)	Time
3	Maximum amplitude	Time
4	Minimum amplitude	Time
5	Peak-to-peak range	Time
6	RMS	Time
7	Signal energy ($\sum x_i^2$)	Time
8	Mean absolute amplitude	Time
9	`argmax` (peak sample index)	Time
10	`argmin` (trough index)	Time
11	Mean FFT magnitude	Spectral
12	Std. FFT magnitude	Spectral
13	Max FFT magnitude	Spectral
14	Spectral peak bin (`argmax` of FFT)	Spectral
15	Spectral energy ($\sum \lvert F_k\rvert^2$)	Spectral
16	Distance (`distance_cm`)	Context

```python
def extract_features(signal):
    fft_mag = np.abs(np.fft.rfft(signal))
    return [
        np.mean(signal),                # mean
        np.std(signal),                 # std
        np.max(signal),                 # max
        np.min(signal),                 # min
        np.ptp(signal),                 # peak-to-peak
        np.sqrt(np.mean(signal**2)),    # RMS
        np.sum(signal**2),              # energy
        np.mean(np.abs(signal)),        # abs_mean
        np.argmax(signal),              # argmax
        np.argmin(signal),              # argmin
        np.mean(fft_mag),               # fft_mean
        np.std(fft_mag),                # fft_std
        np.max(fft_mag),                # fft_max
        np.argmax(fft_mag),             # fft_peak_bin
        np.sum(fft_mag**2),             # fft_energy
    ]
```

Listing 2: Feature extraction (extract_features.py).

Feature redundancy. Pearson correlation analysis of the Day 1 feature matrix reveals severe collinearity: 38 of the 105 possible feature pairs have $|r| > 0.95$, and four pairs are perfectly collinear ($r = 1.000$):

- $\rho(\texttt{std}, \texttt{rms}) = 1.000$ — both measure signal spread; RMS equals std when mean is near zero, as here.
- $\rho(\texttt{std}, \texttt{fft\_std}) = 1.000$ — FFT magnitude spread mirrors time-domain spread under Parseval's theorem.
- $\rho(\texttt{rms}, \texttt{fft\_std}) = 1.000$ — transitively from the above two.
- $\rho(\texttt{energy}, \texttt{fft\_energy}) = 1.000$ — Parseval's theorem: $\sum |x_i|^2 = \sum |X_k|^2 / N$.

Additional near-perfect pairs include $\rho(\texttt{max}, \texttt{ptp}) = 0.999$ and $\rho(\texttt{max}, \texttt{min}) = -0.996$. The effective independent dimensionality of the feature vector is therefore only four to five signals, grouped as: (1) amplitude cluster (`std`/`rms`/`energy`/`fft_max` and their aliases), (2) peak timing (`argmax`/`argmin`), (3) spectral peak location (`fft_peak_bin`), and (4) distance. Feature 14 (`fft_peak_bin`) stands out as the most session-stable: it drifts at most 0.84 % per class between days (see Section V-I).

C. Classifier Training and Storage

The Day 1 feature dataset is split 80/20 with stratification (`random_state=42`), yielding 432 training and 108 test samples. Three classifiers are trained as described in Section II-D:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
import joblib, datetime

ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

models = {
  "Random_Forest": RandomForestClassifier(
      n_estimators=500, random_state=42),
  "SVM": Pipeline([
      ("scaler", StandardScaler()),
      ("clf", SVC(kernel="rbf", C=10,
                  gamma="scale",
                  probability=True))]),
  "KNN": Pipeline([
      ("scaler", StandardScaler()),
      ("clf", KNeighborsClassifier(
                  n_neighbors=7,
                  weights="distance"))]),
}
for name, model in models.items():
    model.fit(X_train, y_train)
    joblib.dump(model, f"models/{name}_{ts}.pkl")
```

Listing 3: Classifier definition and training (train_model.py).

Trained models are serialised with joblib using timestamped filenames so that the most recent version can be loaded deterministically for Day 2 evaluation.

D. Cross-Day Evaluation

Day 2 features are extracted with the identical pipeline. Stored models are loaded and applied to X_{day2} without any retraining or re-scaling — the `StandardScaler` parameters fitted on Day 1 training data are baked into the SVM and KNN pipelines. This deliberately reproduces the deployment scenario in which only a one-time training session is performed.

```
import os, glob, joblib
from sklearn.metrics import accuracy_score,
    classification_report

rf_path = sorted(glob.glob("models/Random_Forest_*.pkl"),
                 key=os.path.getmtime)[-1]
rf_model = joblib.load(rf_path)

X_day2 = df_day2.drop(columns=["label","source_file"])
y_day2 = df_day2["label"]
rf_pred = rf_model.predict(X_day2)

print(accuracy_score(y_day2, rf_pred))
print(classification_report(y_day2, rf_pred))
```

Listing 4: Cross-day evaluation (test_saved_models.py).

E. Discrete-Distance Experiment

A separate model set is trained on Day 1 data filtered to $\{10, 15, 20, 25\}$ cm (referred to as the *4-dist* models). Cross-day evaluation uses the correspondingly filtered Day 2 subset. This experiment tests whether reducing spatial variability during training yields sharper, more transferable decision boundaries.

F. Per-File Majority-Vote Evaluation

To simulate a practical deployment scenario — classifying a complete recording rather than individual waveforms — each source file's five measurements are aggregated by taking the modal predicted class:

```
results = []
for fname in df_eval['source_file'].unique():
    grp = df_eval[df_eval['source_file'] == fname]
    true_lbl  = grp['label'].iloc[0]
    voted_lbl = grp['y_pred'].value_counts().idxmax()
    results.append({'file': fname,
                    'true':  true_lbl,
                    'voted': voted_lbl})
res_df  = pd.DataFrame(results)
overall = (res_df['true'] == res_df['voted']).mean()
```

Listing 5: Per-file majority-vote metric (evaluate_per_file.py).

Day 2 contains 104 unique source files (26 per class), so majority-vote evaluation covers exactly 104 file-level predictions.

G. Result Generation

All figures are produced by `generate_results.py`, which loads serialised models, computes predictions on Day 2 data, and saves publication-quality PNG figures. The 10 generated figures cover overall accuracy, accuracy vs. distance, 4-distance comparisons, confusion matrices (full-range and 4-dist), F1 heatmap, precision/recall per class, temporal drift bar chart, confidence distribution, and per-file majority-vote accuracy.

V. Results and Analysis

A. Day 1 In-Session Performance

Table IV summarises Day 1 held-out test performance (108 samples, 20 % stratified split). Random Forest achieves 82.4 %, clearly outperforming SVM (64.8 %) and KNN (65.7 %). The 5-fold cross-validation scores — RF: 72.0 ± 4.9 %, SVM: 57.8 ± 3.3 %, KNN: 61.1 ± 3.2 % — show that the single-split Day 1 figures carry optimistic bias, and that SVM in particular is notably weaker than its held-out accuracy suggests. The 9.4 pp gap between RF's CV mean and its single-split accuracy is the largest among the three models.

TABLE IV: Day 1 Held-out Test Performance (108 Samples)

Classifier	Evaluation Metrics			
	Accuracy	*Macro Precision*	*Macro Recall*	*Macro F1*
Random Forest	**82.4 %**	0.848	0.823	0.830
SVM	64.8 %	0.657	0.644	0.647
KNN	65.7 %	0.661	0.656	0.656
5-fold CV: RF 72.0 ± 4.9 %, SVM 57.8 ± 3.3 %, KNN 61.1 ± 3.2 %				

B. Temporal Drift — Cross-Day Results

Table V and Fig. 4 summarise performance across all five evaluation scenarios. All classifiers degrade substantially under cross-day conditions, confirming that the feature distribution shifts significantly between sessions.

TABLE V: Classification Accuracy Across Evaluation Conditions

Model	Evaluation Scenarios			
	Day 1	*Day 2 Full*	*D2 4d*[a]	*D2 4d*[b]
Random Forest	82.4	35.3	37.6	**51.8**
SVM	64.8	31.3	32.9	30.6
KNN	65.7	32.3	34.1	35.3
Accuracy drop (Day1 → D2 Full): RF −47.1 pp, SVM −33.5 pp, KNN −33.4 pp				

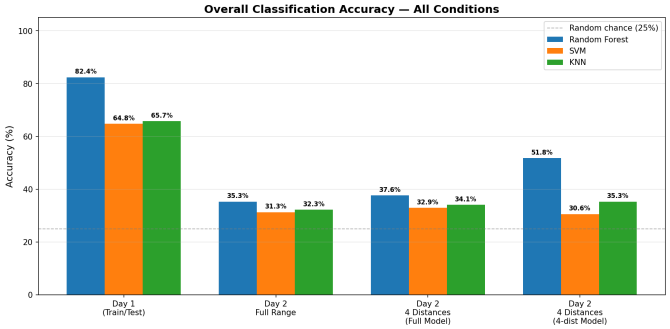

Fig. 4: Overall classification accuracy across all experimental conditions. Random Forest (blue) consistently leads; all models drop

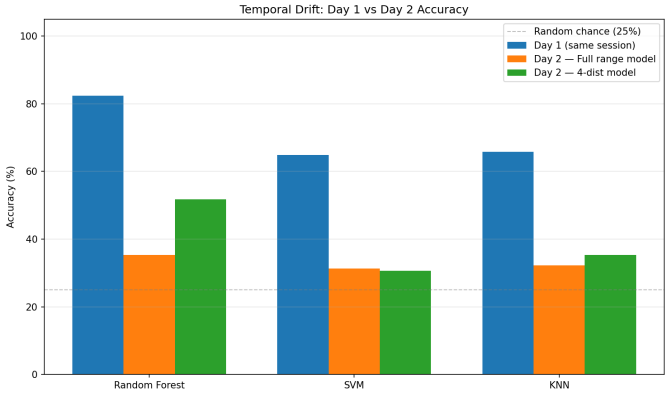

Fig. 5: Temporal drift: Day 1 vs. Day 2 accuracy for all three classifiers. RF drops 47.1 pp, SVM 33.5 pp, KNN 33.4 pp.

C. Accuracy versus Distance

Fig. 6 shows per-distance accuracy for the three full-range models on Day 2 (5–30 cm, $n = 20$ samples per distance except 5 cm and 25 cm where $n = 25$). Performance is highly irregular: at 23 cm RF scores 0 % while at 19 cm and 28 cm it peaks above 65 %. No model maintains uniformly high accuracy across the full range.

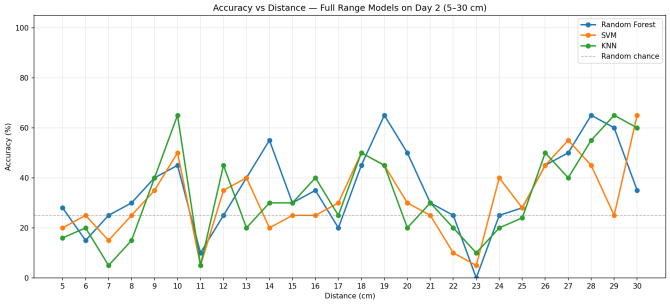

Fig. 6: Accuracy vs. distance — full-range models on Day 2. The irregular pattern reflects the interaction between sensor response and the severity of inter-session drift at each distance.

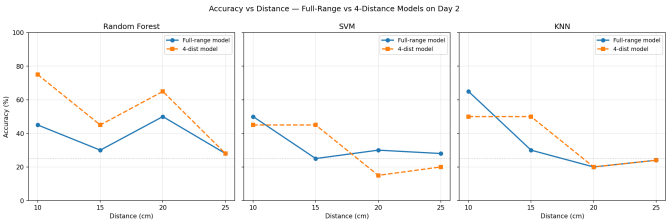

Fig. 7: Comparison of full-range vs. 4-distance models at the four selected distances on Day 2. RF benefits strongly from focused training; SVM and KNN show limited or negative change.

Table VI gives per-distance accuracy for the RF model on Day 2, comparing the full-range and 4-dist models.

TABLE VI: RF Accuracy at Selected Distances (Day 2)

Distance	Model Type	
	Full-range	*4-distance*
10 cm	45.0 %	**75.0 %**
15 cm	30.0 %	45.0 %
20 cm	50.0 %	**65.0 %**
25 cm	28.0 %	28.0 %
30 cm	35.0 %	—

D. Confusion Matrix Analysis

Figs. 8 and 9 show the normalised confusion matrices for all six model configurations on Day 2.

Fig. 8: Normalised confusion matrices — full-range models on Day 2. All models show strongly reduced diagonal dominance; PB w/o Foam is the most confused class across all three.

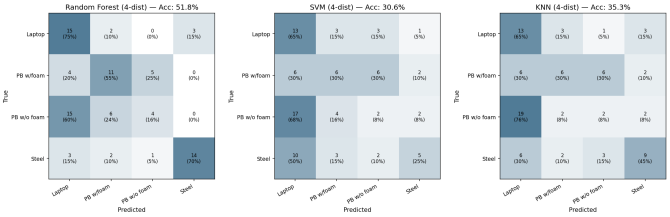

Fig. 9: Normalised confusion matrices — 4-distance models on Day 2. RF (left) shows the strongest diagonal structure; Laptop achieves 75 % recall and Steel achieves 70 % recall.

The RF full-range confusion matrix (absolute counts) is given in Table VII. Each row sums to the number of true samples in that class (130 for Laptop and Steel, 135 for both PB variants on Day 2). The diagonal represents correct predictions.

TABLE VII: RF Full-Range Confusion Matrix — Day 2 (Rows = True, Columns = Predicted)

True Class	Predicted Class			
	Laptop	*PB w/F*	*PB w/oF*	*Steel*
Laptop	**59**	43	12	16
PB w/ Foam	34	**42**	31	28
PB w/o Foam	40	36	**38**	21
Steel	23	41	18	**48**

A notable pattern is that Laptop and Steel are most often confused with PB w/Foam (43 and 41 errors respectively), while PB w/o Foam errors are spread nearly uniformly. This is consistent with the collapse of `argmax` values: on Day 2 Steel's peak shifts from sample 1620 to sample 236, closely matching PB w/Foam's Day 2 mean of sample 724.

E. Per-Class Precision, Recall, and F1

Fig. 10 shows per-class precision, recall, and F1 for all six model configurations on Day 2. Table VIII provides the corresponding numeric values.

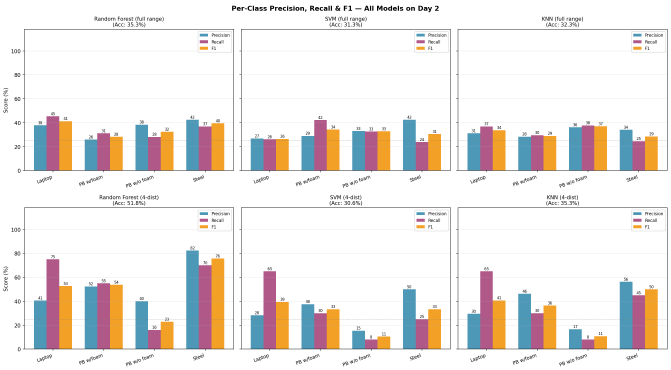

Fig. 10: Per-class precision, recall, and F1 for all six model configurations on Day 2. Top row: full-range models; bottom row: 4-distance models. Each panel reports overall accuracy in the title.

TABLE VIII: Per-class F1 Scores — Day 2 (All Configurations)

Class	Full-Range Model			4-Distance Model		
	RF	*SVM*	*KNN*	*RF*	*SVM*	*KNN*
Laptop	0.413	0.265	0.338	**0.526**	0.394	0.406
PB w/ Foam	0.283	0.343	0.290	**0.537**	0.333	0.364
PB w/o Foam	0.325	0.328	0.370	0.229	0.105	0.108
Steel	0.395	0.305	0.286	**0.757**	0.333	0.500
Macro Avg.	0.354	0.310	0.321	**0.512**	0.291	0.344

Three consistent trends emerge:

1. PB w/o Foam achieves the lowest F1 in every model and condition (range: 0.105–0.370), confirming it as the most challenging class.
2. Steel achieves the highest F1 in the RF 4-dist configuration (F1 = 0.757), reflecting its strong and distinctive high-impedance reflection; however, its F1 is only 0.395 under the full-range model, illustrating how severely drift degrades even the most distinctive class.
3. The RF 4-dist model achieves the highest macro F1 (0.512), but at the cost of near-zero PB w/o Foam F1 (0.229), revealing a class-level trade-off: restricting distances sharpens some boundaries while abandoning others.

F. F1 Heatmap

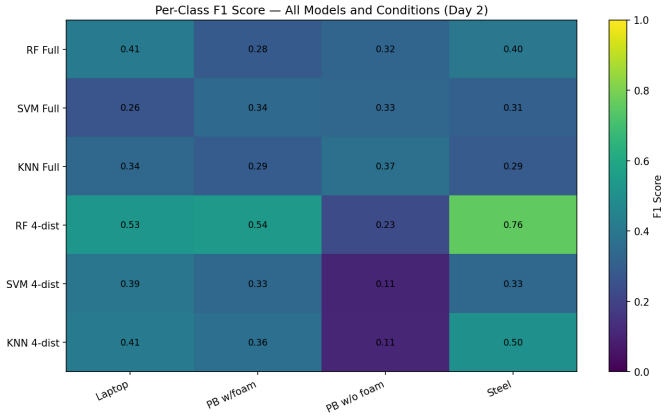

Fig. 11: Per-class F1 heatmap — all six model configurations on Day 2. Darker cells indicate lower F1. PB w/o Foam (third column) is uniformly dark; RF 4-dist (fourth row) is the brightest row overall despite the PB w/o Foam exception.

G. Model Confidence Distribution

Fig. 12 shows the distribution of the maximum predicted probability (confidence) for correct versus incorrect predictions on Day 2.

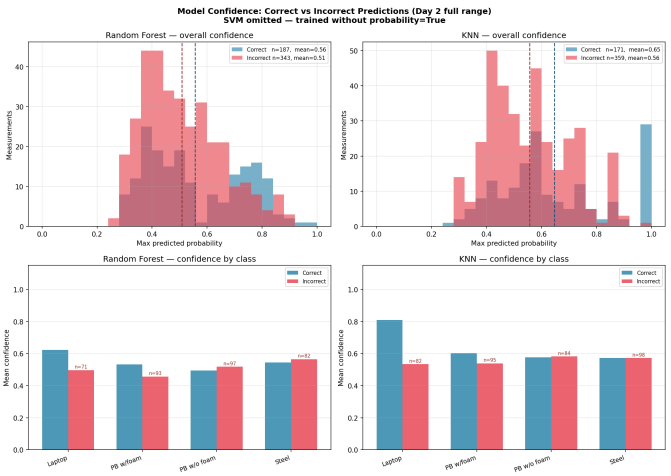

Fig. 12: Model confidence (max predicted probability) for correct vs. incorrect predictions — RF (left) and KNN (right). Top: overall histogram; bottom: mean confidence per class. The small gap between correct and incorrect means reveals weak calibration under drift.

TABLE IX: Model Confidence Statistics — Day 2 (Full Range)

Model	Correct Mean	Incorrect Mean	Gap
Random Forest	0.556	0.508	0.048
SVM	0.499	0.485	0.014
KNN	0.647	0.557	0.090

The SVM gap of 0.014 is negligible: with probability calibration enabled (via `probability=True` in this run), the SVM confidence scores are almost identical whether the prediction is correct or wrong. All three models have substantially weaker calibration than would be expected under matched train/test distributions, confirming that confidence-based rejection thresholds would provide little benefit in a cross-day deployment.

H. Per-File Majority-Vote Accuracy

Fig. 13 and Table X evaluate performance at the file level. Day 2 contains 104 files (26 per class), each with five measurements; the modal predicted class is taken as the file-level prediction.

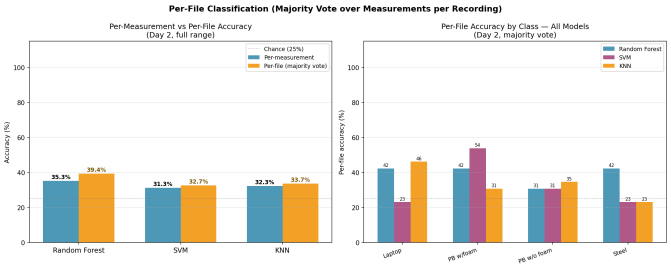

Fig. 13: Per-file (majority-vote) accuracy vs. per-measurement accuracy on Day 2 (left) and per-class breakdown (right). RF gains 4.1 pp from majority voting; SVM and KNN gain less.

TABLE X: Per-file Majority-Vote Accuracy — Day 2 (104 Files)

Model	Overall	Per-Class Accuracy			
		Laptop	*PB w/F*	*PB w/oF*	*Steel*
RF	**39.4 %**	42.3 %	42.3 %	30.8 %	42.3 %
SVM	32.7 %	23.1 %	53.8 %	30.8 %	23.1 %
KNN	33.7 %	46.2 %	30.8 %	34.6 %	23.1 %

The SVM file-level accuracy for PB w/Foam (53.8 %) is substantially higher than its other class scores (23.1 %), indicating a systematic bias toward predicting that class — consistent with the confusion matrix showing that SVM routes many samples from Laptop and Steel to PB w/Foam.

I. Feature Importance and Drift Analysis

Table XI lists the RF feature importances and the per-class inter-session mean shift, computed directly from the Day 1 and Day 2 CSV datasets. `distance_cm` ranks first (9.2 %), consistent with distance being a primary discriminator of echo amplitude.

TABLE XI: RF Feature Importance and Inter-Session Drift (Top 10 Features)

Feature	Importance	Drift per Class (%)			
		Laptop	*PB+F*	*PB-F*	*Steel*
`distance_cm`	0.092	0.0	0.6	1.6	0.5
`abs_mean`	0.086	8.7	14.6	15.3	5.3
`min`	0.081	3.3	7.4	10.5	14.8
`ptp`	0.076	3.0	8.1	10.6	17.9
`fft_max`	0.074	10.7	14.4	16.8	12.1
`argmin`	0.067	76.9	369.6	36.4	87.7
`max`	0.063	2.7	8.8	10.7	20.8
`argmax`	0.063	78.6	569.5	34.2	85.4
`fft_mean`	0.060	7.7	5.8	14.4	18.9
`fft_peak_bin`	0.047	0.3	0.8	0.0	0.8

The contrast between `argmax`/`argmin` and `fft_peak_bin` is striking. Both encode a "peak location", but in entirely different spaces:

- `argmax`/`argmin`: absolute sample index within the 8,192-sample ADC window. Day 1 class means: Steel ≈1620, Laptop ≈1275, PB w/o Foam ≈710, PB w/Foam ≈108. Day 2 class means: Steel ≈236, Laptop ≈273, PB w/o Foam ≈952, PB w/Foam ≈724. The ranking of classes by peak position is almost entirely reversed between sessions.
- `fft_peak_bin`: the dominant frequency component index of the FFT magnitude spectrum. This is determined by the transducer resonance frequency and object impedance, not by sensor position. All four classes drift at most 0.84 % between sessions.

A shift of even a few millimetres in sensor mounting between sessions can advance or retard the echo arrival by hundreds of sample indices, completely reordering the class means of `argmax`. For PB w/Foam the shift reaches 569.5 % (mean moving from sample 108 to sample 724), which causes its Day 2 `argmax` distribution to overlap with the Day 1 distributions of PB w/o Foam and Steel — explaining the high off-diagonal confusion for that class.

VI. Discussion

A. Model Performance Under Distribution Shift

Random Forest (RF) consistently outperforms Support Vector Machine (SVM) and K-Nearest Neighbors (KNN) across all evaluation conditions. Its ensemble-based architecture reduces variance through bootstrap aggregation and mitigates the effect of noisy individual predictions. Furthermore, decision tree splits naturally accommodate correlated and unscaled features, which are prevalent in the extracted feature set.

However, despite these advantages, RF is not robust to inter-session variability. The observed 47.1 pp drop in accuracy between sessions demonstrates that even ensemble methods fail under significant distribution shift. This highlights a key limitation of standard supervised learning pipelines: they implicitly assume that training and test data are drawn from the same distribution.

In contrast, SVM and KNN rely more directly on the absolute feature values at inference time—via margin distances and

nearest-neighbor comparisons, respectively. As a result, even small shifts in feature distributions lead to disproportionately large performance degradation, making these models particularly sensitive to temporal drift.

TABLE XII: Qualitative Comparison of Model Robustness Under Distribution Shift

Model	Performance Characteristics		
	In-session	*Cross-session*	*Drift Robustness*
Random Forest	High	Moderate	Medium
SVM	Moderate	Low	Low
KNN	Moderate	Low	Low

B. Distance-Specific Training

Restricting training to a subset of distances provides measurable benefit only for RF, improving performance by +14.1 pp to 51.8 %. This suggests that RF is capable of exploiting reduced intra-class variability to learn more stable decision boundaries within a constrained feature space.

In contrast, SVM and KNN do not benefit from this restriction. This behaviour is consistent with their reliance on precise feature geometry: reducing the diversity of the training set does not compensate for the mismatch between training and testing distributions. These findings indicate that model capacity alone is insufficient; robustness to distribution shift must be explicitly engineered.

C. Mechanisms of Temporal Drift

The severe performance degradation (47 pp for RF) can be attributed to three interacting mechanisms:

1. **Position-dependent features.** The features `argmax` and `argmin` directly encode the temporal position of waveform peaks. Small physical shifts in sensor placement (on the order of millimetres) translate into large index shifts (hundreds of samples), causing substantial feature drift.
2. **Amplitude sensitivity.** Environmental factors such as temperature and humidity influence the speed of sound and attenuation characteristics. This results in systematic variation in signal amplitude, affecting all amplitude-derived features and introducing session-dependent scaling effects.
3. **Feature redundancy.** A large subset of the feature space exhibits near-perfect correlation ($r > 0.95$). This redundancy reduces the effective dimensionality and amplifies noise: perturbations in one feature propagate across multiple correlated features, destabilising the learned decision boundaries.

D. Class Separability

Class-wise analysis reveals systematic differences in separability. The two plastic-box variants (with and without foam) remain the most challenging pair across all conditions. Their similar structural properties produce overlapping acoustic signatures, with the foam insert introducing only subtle attenuation and diffusion effects. These differences become increasingly difficult to detect at larger distances due to reduced signal-to-noise ratio.

In contrast, the steel class is consistently the easiest to classify. Its high acoustic impedance produces strong reflections and a distinct spectral signature, resulting in stable feature representations even under distribution shift. This is reflected in consistently higher F1-scores across all evaluation settings.

E. Limitations

Several limitations of this study should be acknowledged:

- **Limited temporal scope.** The analysis is based on only two sessions, making it difficult to distinguish systematic drift from session-specific artefacts. Additional sessions are required for statistically robust conclusions.
- **Single train/test split.** In-session performance (82.4 %) is derived from a fixed 80/20 split rather than cross-validation, which may introduce variance in the reported estimate.
- **Lack of environmental metadata.** Temperature and humidity were not recorded, preventing direct correlation between environmental conditions and observed signal drift.
- **Incomplete probabilistic analysis.** The SVM model was trained without enabling probabilistic outputs (`probability=True`), limiting comparative calibration analysis.
- **Underutilised signal representations.** Although FFT waveforms are extracted during preprocessing, they are not incorporated into feature engineering, representing a missed opportunity for capturing frequency-domain invariants.

F. Recommendations for Improvement

Based on the findings, the following improvements are prioritised:

a) Tier 1 — Address feature-level drift.: Remove or replace `argmax` and `argmin` with invariant alternatives. Emphasise frequency-domain descriptors such as dominant frequency bins, spectral centroid, zero-crossing rate, and band-energy ratios. Fully utilise the available FFT data to construct more robust representations.

b) Tier 2 — Improve evaluation methodology.: Adopt leave-one-session-out cross-validation as the primary evaluation protocol. Incorporate systematic feature ablation studies and correlation analysis. Enable probability estimation for SVM and extend the model comparison to include gradient boosting and neural network approaches.

c) Tier 3 — Extend analytical depth.: Quantify distribution shift using statistical tests (e.g., Kolmogorov–Smirnov test) applied per feature and class. Visualise feature drift through violin or kernel density plots. Establish a baseline for domain adaptation using methods such as covariance alignment (CORAL) or feature normalisation techniques.

Overall, the discussion demonstrates that the dominant challenge in ultrasonic object classification is not model selection but robustness to distribution shift. Addressing feature invariance and incorporating drift-aware methodologies are essential steps toward reliable real-world deployment.

VII. Conclusion

This study systematically investigated the applicability of machine learning for cross-session ultrasonic object classification within a controlled optical bench setup. Three widely used classifiers—Random Forest (RF), Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)—were evaluated across five experimental conditions spanning two separate measurement sessions.

The results highlight a critical gap between in-session performance and real-world generalisation. While Random Forest achieves a strong in-session accuracy of 82.4 % on Day 1, its performance drops sharply to 35.3 % on Day 2, demonstrating that models trained under static conditions significantly overestimate their robustness. This degradation is consistently observed across all classifiers, confirming the presence of substantial temporal drift. Among the evaluated methods, RF exhibits comparatively greater robustness, whereas SVM and KNN show higher sensitivity to distributional changes.

Targeted mitigation strategies reveal mixed effectiveness. Distance-specific retraining yields a notable improvement for RF (+14.1 percentage points to 51.8 %), suggesting that partial adaptation to environmental variation can recover performance. However, similar gains are not observed for SVM and KNN, indicating model-dependent sensitivity to retraining strategies. Additionally, per-file majority voting provides only marginal improvement (RF: 35.3 % $\rightarrow$ 39.4 %), and confidence calibration analysis shows that predictive probabilities become unreliable under drift conditions (RF calibration gap: 0.048).

A key contribution of this work is the identification of the underlying cause of performance degradation. Post-hoc feature analysis reveals that the features `argmax` and `argmin`, which encode absolute peak positions within the waveform window, are the dominant drivers of drift. These features exhibit substantial shifts (39–570 % per class) between sessions due to minor sensor repositioning, effectively introducing session-specific bias. This finding underscores the importance of feature invariance in ultrasonic signal processing pipelines.

From a class-wise perspective, the results further indicate that classification difficulty is not uniform: *PB without foam* consistently represents the most challenging class, while *Steel* remains the most reliably identifiable, suggesting inherent differences in signal separability.

Overall, this study demonstrates that ultrasonic classification systems trained on single-session data are not suitable for deployment without explicit handling of domain shift. The findings strongly advocate for the adoption of drift-aware design strategies, including periodic retraining, domain adaptation techniques, and, most critically, the removal of session-dependent features. Replacing `argmax`/`argmin` with more robust, session-invariant spectral or statistical descriptors represents the most promising direction for improving cross-session generalisation.

Future work should explore advanced approaches such as feature normalisation, invariant representation learning, and domain generalisation methods, as well as validation under more diverse environmental conditions. These steps are essential to bridge the gap between controlled experimental performance and reliable real-world deployment.

Acknowledgement

The authors express their sincere gratitude to **Prof. Dr. Andreas Pech** of Frankfurt University of Applied Sciences for his outstanding supervision, continuous encouragement, and constructive feedback throughout the Machine Learning course in Winter Semester 2025/26. His clearly structured lectures provided a rigorous theoretical foundation that shaped every stage of this experimental work. His thoughtful guidance on project methodology, patient review of results, and insightful comments on the interpretation of temporal drift were instrumental in shaping the depth and quality of this paper. His genuine commitment to student development and enthusiasm for applied machine learning left a lasting impression on our approach to research.

We also warmly thank **Julian Umansky** for generously providing access to the Red Pitaya Ultrasonic Measurement System and for invaluable technical support during the experimental setup. His hands-on assistance configuring the Frankfurt UAS UDP client, explaining the Red Pitaya STEMlab hardware architecture, and advising on measurement protocol was essential to the successful acquisition of the two-day dataset underpinning all results in this paper.

This project was completed as part of the Machine Learning course, M.Eng. in Information Technology, Frankfurt University of Applied Sciences, Germany, Winter 2025/26.

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, New York, 2006.

[2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, New York, 2009.

[3] J. Blitz, *Fundamentals of Ultrasonics*. New York, NY, USA: Plenum Press, 1967.

[4] Red Pitaya, "Red Pitaya Documentation," https://redpitaya.com/documentation, 2024.

[5] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media, Sebastopol, 2019.

[6] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, Upper Saddle River, 2007.

[7] P. Mehta *et al.*, "A high-bias, low-variance introduction to machine learning for physicists," *Physics Reports*, vol. 810, pp. 1–124, 2019.

[8] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, Upper Saddle River, 2009.

[9] B. Sun and K. Saenko, "Deep CORAL: Correlation alignment for deep domain adaptation," in *Proc. ECCV Workshops*, Amsterdam, 2016, pp. 443–450.